Hopf algebra basics: rooted trees

You don't need to read the following notes by Foissy, but I refer to them a few times here, for completeness:
http://loic.foissy.free.fr/pageperso/preprint3.pdf

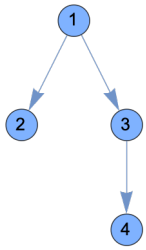
In a way, what we do here is the "complement" of what we did with the symmetric group, where we focused on the *cycles* (think of the Cayley graph). Here we focus on **graphs *without cycles, called trees***. They are also a special kind of tree graphs, trees that have a root.

In[1766]:=
```
Remove["Global`*"] // Quiet
```

Here's the example in Foissy:

In[1767]:=
```
example = TreeGraph[{1 ↔ 2, 1 ↔ 3, 3 ↔ 4},
  VertexLabels → Placed["Name", Center], VertexSize → 0.3]
```

Out[1767]=

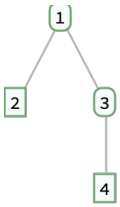


This is a `Graph` data structure, as in the basic group file.
We would like to **cut an edge** in the graph (=one of the arrows) and see what "falls down".

Actually, `Tree` is its own data structure in computer science more generally. Think an actual tree with root and branches, but *upside down*. Or, a *family tree* going downwards, with grandparent, children, and so on):

In[1768]:=
```
exampleTree = GraphTree[%]
```

Out[1768]=



The top node is the **root**. (Unlike graphs, Mathematica draws the bottom nodes in trees with a slightly different symbol, like 4, but that's not important here.)
Internally a tree is represented as:
(for more details on this data structure, see below)

In[1769]:=
```
InputForm[%]
```

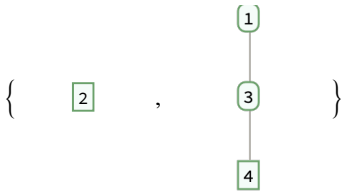
Out[1769]//InputForm=
```
Tree[1, {Tree[2, None], Tree[3, {Tree[4, None]}]}]
```

where "`None`" means "no child" (using the terminology of *family* trees).

Now, to make a bialgebra, we cut the tree branches, and keep the fallen pieces separate from the root-connected piece by the tensor product symbol ⊗. An **admissible cut** is one where no path through the original tree meets more than one cut.

Cut the edge 1 ⟷ 2, and put the "pruned" piece that falls down first:

In[1770]:=
```
cut1 = {Tree[2, None], TreeDelete[exampleTree, 1]}
```
Out[1770]=



This is number 4 in Foissy's table.
It's not admissible to cut this again.
Now cut the edge 1 ⟷ 3:

In[1771]:=
```
cut2 = {TreeChildren[exampleTree][[2]], TreeDelete[exampleTree, {2}]}
```
Out[1771]=



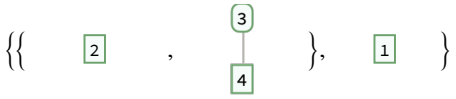
The above is number 2 in Foissy's table.
Disconnected parts are allowed, so we can further cut the edge 1 ⟷ 2 above:

In[1772]:=
```
cut3 = {Tree[3, {Tree[4, None]}], {Tree[1, None], Tree[2, None]}}
```
Out[1772]=



It can also be grouped like this.

In[1773]:=
```
cut4 = {{Tree[2, None], Tree[3, {Tree[4, None]}]}, Tree[1, None]}
```
Out[1773]=



Note that cutting the edge 3 ⟷ 4 in these last things is not admissible anymore, since we already cut one edge on the same path: 1 ⟷ 3.

There is now only one admissible cut left, that I leave for you.

The bialgebra is e ⊗ t + t ⊗ e + (what falls down ⊗ what is left), where "e" is the empty tree.

This bialgebra becomes Hopf by finding an “inverse”, the antipode *s*.

First for just a single node •, call it *a*.

In[1774]:=
```
Δa = e ⊗ a + a ⊗ e ;
SIdΔa = S[e] ⊗ a + S[a] ⊗ e;
ruleμ = {(a_ ⊗ b_) → a b}; ruleS = S[e] → e;
SIdΔa /. ruleS;
% //. ruleμ
% == 0 /. e → 1
Solve[%, S[a]] // First
```

Out[1778]=
$$e\, S(a) + a\, e$$

Out[1779]=
$$S(a) + a = 0$$

Out[1780]=
$$\{S(a) \to -a\}$$

OK. Now the tree with two nodes, call it *t.* As is intuitive, cutting it produces two pieces $a \otimes a$, where again a = •.
The coproduct is:

In[1781]:=
```
Δt = e ⊗ t + t ⊗ e + a ⊗ a;
SIdΔt = S[e] ⊗ t + S[t] ⊗ e + S[a] ⊗ a;
ruleS = {S[e] → e, S[a] → -a};
SIdΔt /. ruleS
% //. ruleμ
% == 0 /. e → 1
Solve[%, S[t]] // First
```

Out[1784]=
$$-a \otimes a + S(t) \otimes e + e \otimes t$$

Out[1785]=
$$-a^2 + e\, S(t) + e\, t$$

Out[1786]=
$$-a^2 + S(t) + t = 0$$

Out[1787]=
$$\{S(t) \to a^2 - t\}$$

There is a general formula in Foissy, which this can be compared to.
Now some more generic but related comments.

We can construct binary operations by **Groupings**.
For example, on four ordered elements *a,b,c,d* we have:

In[1788]:=
```
Groupings[{a, b, c, d}, CircleTimes → 2]
```

Out[1788]=
$$\{((a \otimes b) \otimes c) \otimes d,\ a \otimes ((b \otimes c) \otimes d),\ (a \otimes (b \otimes c)) \otimes d,\ a \otimes (b \otimes (c \otimes d)),\ (a \otimes b) \otimes (c \otimes d)\}$$

Take the first term. It can be represented as a tree:

In[1789]:=
```
example = %[[1]]
exampleTree = TreeForm[example]
```

Out[1789]=
$((a \otimes b) \otimes c) \otimes d$

Out[1790]//TreeForm=

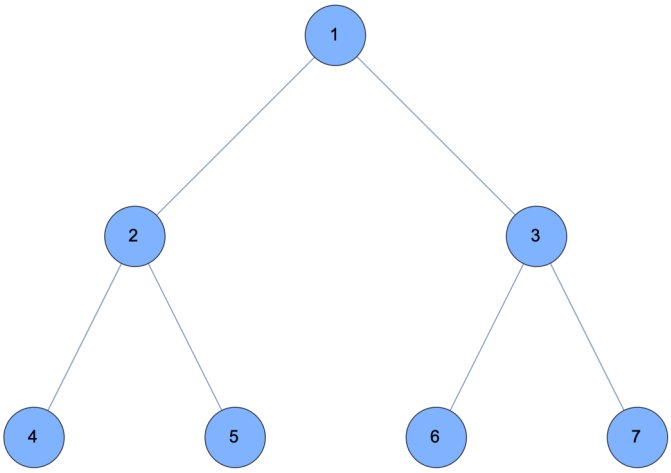


At first, it's easiest to read this from bottom to top.
The importance of this is that symbolic manipulation software like Mathematica in fact store mathematical expressions as trees internally.

Use one of the predefined graphs as example, here the *complete Kary tree* (the name doesn't matter, it's pretty obvious what it is):

In[1791]:=
```
biggerExample =
 CompleteKaryTree[3, VertexLabels → Placed["Name", Center], VertexSize → 0.3]
```

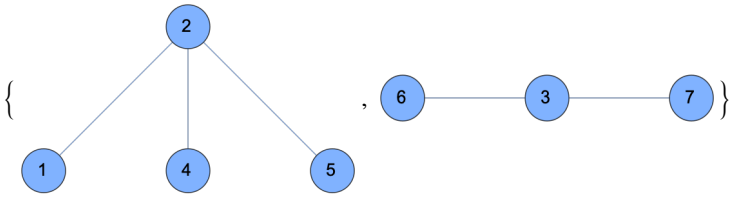
Out[1791]=



As mentioned above, the "lines" in this **Tree** data structure aren't their own objects, so it's more natural to cut edges in the corresponding **Graph**:

In[1792]:=
```
ConnectedGraphComponents[EdgeDelete[biggerExample, {1 ⟼ 3}]]
```

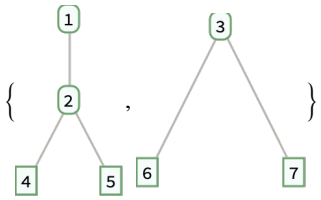
Out[1792]=



Again, **Graphs** don't have a root, but we can easily display them as **Trees**:

In[1793]:=
```
Map[GraphTree, %]
```

Out[1793]=



To get the above Hopf algebra, these have to be ordered.

Now two assorted facts.

• The Kirchhoff matrix (same as in the circuit laws):

In[1794]:=
```
KirchhoffMatrix[biggerExample] // Normal
```

Out[1794]=

$$\begin{pmatrix} 2 & -1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 3 & 0 & -1 & -1 & 0 & 0 \\ -1 & 0 & 3 & 0 & 0 & -1 & -1 \\ 0 & -1 & 0 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

• One recent application is solvable quantum circuits:
https://arxiv.org/abs/2409.17215